

The Kind of Kubernetes: Using Docker Desktop for Reproducible Kubernetes Clusters

Namdak Tonpa

July 8, 2026

Contents

1	Introduction	1
1.1	Architectural Analysis	2
2	Kind of Kubernetes	2
2.1	Prerequisites	2
2.2	Installation	2
2.2.1	On macOS	2
2.2.2	On Linux	3
2.3	Using Kind with Docker Desktop	3
2.4	Cluster Provisioning via Docker Desktop and CLI	3
2.5	Recommended kind-config.yaml for ERP/1	4
2.6	Deployment of ERP/1 Components	4
2.7	Node Specialization	4
2.8	Best Practices	5
2.9	Migration to Production Environments	5
2.10	Managing Clusters with kind.sh	5
3	Backup and Restore	6
3.1	Backup and Restore with Device-Level Snapshots	6
3.1.1	Technical Architecture of backup.sh	7
3.1.2	Restore Semantics (restore.sh)	7
3.1.3	Interactive Exploration via view.sh	8
3.1.4	Mission Alignment & Production Fidelity	8
4	Image Management	9
4.1	Local Docker Desktop Builder (images.sh --local)	9
4.2	Building with In-Cluster Registry (--registry)	9
4.3	Best Practices for Reproducibility	10
5	Conclusion	10

1 Introduction

Kind (Kubernetes IN Docker) constitutes a precise instrument for instantiating standards-compliant Kubernetes clusters wherein each node is realized as a Docker container.

For the development of ERP/1, the combination of Kind and Docker Desktop yields a controlled environment that closely approximates production conditions on both Linux and macOS systems. The present document provides a rigorous exposition of the configuration and utilization of this system.

Kind was introduced in 2018¹ and employs the official `kubeadm` bootstrap mechanism, ensuring semantic equivalence with production Kubernetes deployments at the level of the control plane and worker node behavior.

1.1 Architectural Analysis

The system operates by constructing a graph of Docker containers, each executing a full `kubelet` and associated Kubernetes components. Cluster initialization follows the canonical `kubeadm init` and `kubeadm join` protocol.

This design confers two principal advantages: (i) exact reproducibility of Kubernetes semantics, and (ii) minimal deviation from the production execution model. Consequently, scheduling, networking, storage, and security primitives behave in a manner that is formally transferable to bare-metal or cloud-orchestrated clusters.

2 Kind of Kubernetes

2.1 Prerequisites

The following conditions must be satisfied:

- Docker Desktop (version ≥ 4.38) with containerd image store enabled.
- Minimum 8 GB RAM (16 GB or higher recommended for multi-node configurations hosting ERP/1 workloads).
- Hardware-assisted virtualization support.
- Working installation of `kubect1` and Helm.

2.2 Installation

2.2.1 On macOS

```
brew install kind
```

¹The project Kind was started by Ben Elder (a Kubernetes maintainer) and quickly became very popular in the community because it is lightweight, fast, and great for local development and testing. By 2019–2020, it had gained massive adoption and is now one of the most widely used tools for local Kubernetes clusters.

2.2.2 On Linux

Download the binary or use package managers:

```
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.32.0/kind-linux-amd64
chmod +x ./kind
sudo mv ./kind /usr/local/bin/kind
```

2.3 Using Kind with Docker Desktop

Docker Desktop implements Kind as a native provisioner. For reproducible deployments, the command-line interface with an explicit configuration file is preferred. A recommended configuration for ERP/1 workloads is as follows:

1. Open Docker Desktop Settings → Kubernetes
2. Enable Kubernetes and select **Kind** as the provisioner
3. Create a multi-node cluster

Alternatively, use the CLI for more control:

2.4 Cluster Provisioning via Docker Desktop and CLI

Docker Desktop implements Kind as a native provisioner. For reproducible deployments, the command-line interface with an explicit configuration file is preferred. A recommended configuration for ERP/1 workloads is as follows:

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: erp1-local
nodes:
  - role: control-plane
  - role: worker
  - role: worker
kubeadmConfigPatches:
  - |
    kind: ClusterConfiguration
    networking:
      podSubnet: "10.244.0.0/16"
```

Cluster creation command:

```
kind create cluster --config kind-config.yaml --name erp1-local
```

2.5 Recommended kind-config.yaml for ERP/1

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: erp1-prod-like
nodes:
  - role: control-plane
  - role: worker
  - role: worker
kubeadmConfigPatches:
  - |
    kind: ClusterConfiguration
    networking:
      podSubnet: "10.244.0.0/16"
containerdConfigPatches:
  - |-
    [plugins."io.containerd.grpc.v1.cri".registry.mirrors]
    [plugins."io.containerd.grpc.v1.cri".registry.mirrors."docker.io"]
      endpoint = ["https://registry-1.docker.io"]
```

2.6 Deployment of ERP/1 Components

Integration with the repository tooling proceeds as:

```
./prereq.sh
kind create cluster --config kind-config.yaml
./deploy.sh
```

The multi-node topology permits accurate modeling of pod scheduling, inter-service communication, and resource isolation.

2.7 Node Specialization

Critical services such as `ns-dns` and `ca-pki` benefit from explicit node dedication. Labeling and affinity rules are applied as follows:

```
kubectl label node <worker-name> dedicated=dns
kubectl label node <worker-name> dedicated=pki
```

Subsequent Deployments should specify `nodeSelector` or `nodeAffinity` fields to enforce placement constraints. This practice mirrors production hardening strategies and facilitates early detection of resource contention.

Key advantages for ERP/1:

- Multi-node setup mimics production topology
- Fast iteration cycle

- Full support for Helm, Operators, and complex dependencies (ns-dns, capi, etc.)
- Easy node dedication using labels and affinity

2.8 Best Practices

- Maintain strict version alignment between local Kind clusters and target production environments.
- Prefer containerd over Docker shim for improved isolation and performance.
- Systematically export manifests via `kubectl get --all-namespaces -o yaml` for GitOps synchronization.
- Employ Kustomize overlays to manage environment-specific variations.
- Continuously monitor resource utilization through `kubectl top` and Docker Desktop metrics.

2.9 Migration to Production Environments

Owing to Kind's use of unmodified Kubernetes components, the transition of manifests to production clusters requires only localized modifications, primarily concerning persistent storage (replacement of `hostPath` with CSI volumes) and ingress/load-balancing configuration.

2.10 Managing Clusters with `kind.sh`

For convenient management of multiple local clusters alongside Docker Desktop, we provide a custom wrapper script `kind.sh`.

```
#!/bin/bash

set -euo pipefail

NAME="${2:-desktop}"
CLUSTER_NAME="$NAME"

create() {
  echo "Creating Kind cluster: $CLUSTER_NAME"
  kind create cluster --name "$CLUSTER_NAME" --wait 2m
  kind get kubeconfig --name "$CLUSTER_NAME" > /tmp/kc.yaml
  KUBECONFIG=~/.kube/config:/tmp/kc.yaml \
  kubectl config view --flatten > ~/.kube/config.new
  mv ~/.kube/config.new ~/.kube/config
  rm -f /tmp/kc.yaml
}
```

```

    echo "Cluster $CLUSTER_NAME created"
}

delete() {
    echo "Deleting cluster: $CLUSTER_NAME"
    kind delete cluster --name "$CLUSTER_NAME" 2>/dev/null || true
    kubectl config delete-context "kind-$CLUSTER_NAME" 2>/dev/null || true
    kubectl config delete-cluster "$CLUSTER_NAME" 2>/dev/null || true
    kubectl config delete-user "$CLUSTER_NAME" 2>/dev/null || true
    echo "Deleted $CLUSTER_NAME"
}

list() {
    echo "=== KinD Clusters ==="
    kind get clusters || echo "None"
    echo ""
    echo "=== Kubectl Contexts ==="
    kubectl config get-contexts
}

case "${1:-list}" in
    create) create ;;
    delete) delete ;;
    list) list ;;
    *) echo "Usage: ./kind.sh <create|delete|list> [name]" ;;
esac

```

This script allows clean management of multiple environments:

```

./kind.sh create synrc
./kind.sh kind
./kind.sh list
./kind.sh delete synrc
./kind.sh docker

```

3 Backup and Restore

3.1 Backup and Restore with Device-Level Snapshots

A core mission of ERP/1 infrastructure is complete reproducibility and stateful resilience. Local Kind clusters must support not only rapid iteration but also faithful simulation of production disaster-recovery and data-migration workflows. The repository delivers this through a set of purpose-built shell scripts that perform device-level filesystem snapshots of all PersistentVolumeClaims (PVCs) and associated StatefulSets.

```

./backup.sh          # Full device-level backup
./restore.sh <dir>   # Targeted restore from backup
./view.sh <dir>      # Inspect without extraction

```

These tools operate directly against the Kind control-plane container (typically `synrc-control-plane`), leveraging the `hostPath` volumes exposed by Docker Desktop/Kind for bit-exact data capture.

3.1.1 Technical Architecture of backup.sh

The backup process follows a rigorously safe, atomic sequence:

1. Manifest Export & Sanitization

Exports `statefulset`, `deployment`, `pvc`, `service`, and `configmap` resources across ERP/1 namespaces (`erp-infra`, `erp-telemetry`, `erp-security` etc.). A Ruby filter `manifest.rb` strips transient metadata (UIDs, `resourceVersions`, `creationTimestamps`) to produce clean, re-applicable YAML.

2. Graceful Pod Quiescence

Scales all `StatefulSets` and `PVC`-attached `Deployments` to 0 replicas. This guarantees volumes are unmounted, eliminating filesystem inconsistency during snapshotting.

3. Device-Level Snapshotting

For each `PVC`:

- Resolves the backing `PersistentVolume` `hostPath`.
- Executes `docker exec synrc-control-plane tar -czf - ...` directly inside the Kind node.
- Stores compressed tarballs named `<namespace>@<pvc-name>.tar.gz`.

This produces portable, mountable filesystem images preserving ownership, permissions, extended attributes, and directory structure.

4. Metadata & Verification

Generates `BACKUP_INFO.txt` with timestamps, `PVC` capacities, file counts, and sizes. Restarts pods to their original replica counts.

Output resides in `./priv/YYYYMMDD-HHMMSS/` — a timestamped, self-contained backup bundle.

3.1.2 Restore Semantics (restore.sh)

Restore mirrors backup in reverse, with additional safety guarantees:

1. Apply sanitized manifests (force-conflicts to overwrite cleanly).
2. Wait for dynamic `PVC` provisioning and binding.

3. Scale down pods again.
4. For each backup tarball:
 - Parse `namespace@pvc` naming convention.
 - Clear target `hostPath` directory.
 - Stream-extract with `tar -xzf --strip-components=1` directly into the volume.
5. Scale pods back up and provide verification (`kubect1 get pvc,pods`).

3.1.3 Interactive Exploration via `view.sh`

The companion viewer supports non-destructive inspection:

- Summary of backup metadata and PVC inventory.
- `list <image>` — contents of a tarball.
- `tree <image>` — hierarchical directory visualization.
- `cat <image> <path>` — peek at individual files.
- `extract <image> <outdir>` — targeted filesystem extraction.

This capability is invaluable for debugging data corruption, auditing persisted state, or performing selective recovery.

3.1.4 Mission Alignment & Production Fidelity

By operating at the raw device/filesystem layer (rather than application-level export), these tools achieve near-perfect fidelity to bare-metal or cloud CSI-based backups. They eliminate the semantic gap between development and production storage behavior — a foundational requirement for ERP/1's "develop once, deploy anywhere" philosophy. Combined with 'kind.sh' cluster lifecycle management, they enable reproducible testing of backup windows, restore drills, and data-migration scenarios with zero external dependencies.

Recommended workflow for ERP/1 developers:

```
./prereq.sh && ./kind.sh create synrc && ./kind.sh kind
./deploy.sh
./backup.sh
./delete-pvc.sh
./restore.sh ./priv/20260708-002817/
```

4 Image Management

Maintaining reproducible container images is fundamental to ERP/1's "develop once, deploy anywhere" mission. The tooling supports two complementary strategies that work seamlessly with Docker Desktop + Kind:

- **In-cluster Docker Registry** (production-like, via `docker-registry` service)
- **Local Docker Builder** (fast, registry-free development workflow)

4.1 Local Docker Desktop Builder (`images.sh --local`)

The script `images.sh --local` provides deep visibility into the Docker Desktop environment:

```
./images.sh --local
```

It reports:

- Active `buildx` builders (including default Linux builder)
- Local `erpuno/*` images in the Docker daemon
- Images loaded into Kind nodes via `crictl`

This mode is ideal for developers who prefer building directly with Docker's native builder (leveraging BuildKit) and loading images into Kind without pushing to a registry.

Workflow for registry-free development:

```
docker buildx build --load -t erpuno/my-service:latest ./path
kind load docker-image erpuno/my-service:latest --name erp1-local
```

This approach offers maximum speed and simplicity for local iteration while remaining fully reproducible across Linux/macOS/Windows via Docker Desktop.

4.2 Building with In-Cluster Registry (`--registry`)

Component-level `build.sh` scripts (invoked via `rebuild.sh`) support a `--registry` flag (or equivalent) to push images to the in-cluster `docker-registry:5000` service.

Typical pattern inside component `build.sh`:

```
REGISTRY="docker-registry.erp-infra.svc.cluster.local:5000"
docker buildx build \
  --push \
  --platform linux/amd64 \
  -t ${REGISTRY}/erpuno/my-service:${TAG} .
```

Full rebuild cycle:

```
./rebuild.sh      # resets registry + rebuilds + deploys  
./images.sh      # inspect registry catalog
```

The in-cluster registry (exposed on port 5000) mirrors production image distribution practices and enables multi-node Kind clusters to pull images consistently.

4.3 Best Practices for Reproducibility

- Use `images.sh --local` for rapid development cycles on a single node.
- Switch to `--registry` mode when testing multi-node scheduling or production image pull behavior.
- Pin image tags and use `buildx` with explicit platforms (`linux/amd64`) for cross-environment consistency.
- Combine with Kind's `--config` for custom containerd mirrors when needed.
- Monitor builder state and registry health via Docker Desktop UI + `kubect1` port-forward.

This dual-path strategy eliminates friction for developers while preserving semantic equivalence with production Kubernetes image management — a key pillar of ERP/1 operational excellence.

5 Conclusion

Kind, when orchestrated through Docker Desktop, provides a mathematically clean and operationally efficient platform for local validation of ERP/1 systems. Its fidelity to the Kubernetes specification renders it a rational choice for developers seeking to minimize the semantic gap between development and production.